

Greedy Algorithms, Part 1

Files on Tape, Huffman Coding

Greedy Algorithms

Build a solution through a series of steps, each of which takes the choice that looks best **right now**.

Greedy Algorithms

Build a solution through a series of steps, each of which takes the choice that looks best **right now**.



Greedy Algorithms

Build a solution through a series of steps, each of which takes the choice that looks best **right now**.

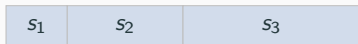


The hard part is never the algorithm. It is proving the local choice is globally optimal.

Storing Files on Tape

The Problem

n files with sizes $s_1, \dots, s_n$ sit on a tape. Tape access is **sequential**: reading a file means reading everything before it first.



to read s_3 you must pass s_1 and s_2

If the files are stored in order σ , the cost of retrieving $\sigma(j)$ is

$$\text{cost}(\sigma(j)) = \sum_{k=1}^j s_{\sigma(k)}.$$

Order Matters

Three files with sizes [1, 2, 3].

[1, 2, 3]:

1	2	3
---	---	---

 costs 1, 3, 6 total = 10

[3, 2, 1]:

3	2	1
---	---	---

 costs 3, 5, 6 total = 14

Same files, different total cost. Which order is best?

Total Retrieval Cost

Assuming every file is equally likely to be requested,

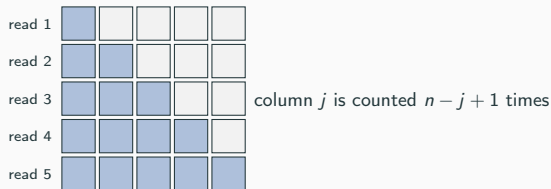
$$C(\sigma) = \sum_{j=1}^n \sum_{k=1}^j s_{\sigma(k)} = \sum_{j=1}^n (n - j + 1) s_{\sigma(j)}.$$

Total Retrieval Cost

Assuming every file is equally likely to be requested,

$$C(\sigma) = \sum_{j=1}^n \sum_{k=1}^j s_{\sigma(k)} = \sum_{j=1}^n (n - j + 1) s_{\sigma(j)}.$$

Why the second form? File $\sigma(j)$ is read for its own retrieval and for every file after it:



The Greedy Rule

Claim

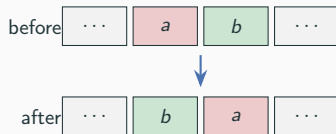
Sorting the files in **non-decreasing order of size** is optimal.

Intuition. A file placed early adds its size to the cost of every later retrieval. So the ones we pay for most often should be the **small** ones.

The rule is obvious. The proof is the interesting part.

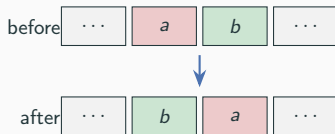
Exchange Argument

Suppose an optimal order has adjacent files a then b with $s_a > s_b$. Swap them.



Exchange Argument

Suppose an optimal order has adjacent files a then b with $s_a > s_b$. Swap them.



- Reading a costs s_b **more** (now b comes first).
- Reading b costs s_a **less** (it no longer skips a).
- Every other file is unaffected.

Exchange Argument

The net change in total cost is

$$s_b - s_a < 0 \quad \text{whenever } s_b < s_a.$$

So the swap **strictly improves** the order, contradicting optimality.

Hence no optimal order has a larger file immediately before a smaller one, which means the optimal order is sorted by non-decreasing size.

Sorting dominates: $O(n \log n)$.

Files with Access Probabilities

A Twist

Now file i also has an access probability $p_i > 0$, with $\sum_i p_i = 1$. The **expected** retrieval cost is

$$C(\sigma) = \sum_{j=1}^n p_{\sigma(j)} \sum_{k=1}^j s_{\sigma(k)}.$$

A Twist

Now file i also has an access probability $p_i > 0$, with $\sum_i p_i = 1$. The **expected** retrieval cost is

$$C(\sigma) = \sum_{j=1}^n p_{\sigma(j)} \sum_{k=1}^j s_{\sigma(k)}.$$

Example. $(s_1, p_1) = (10, 0.1)$ and $(s_2, p_2) = (1, 0.9)$.

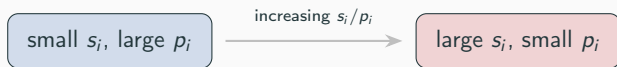
order	expected cost
[1, 2]	$0.1 \cdot 10 + 0.9 \cdot (10 + 1) = 10.9$
[2, 1]	$0.9 \cdot 1 + 0.1 \cdot (1 + 10) = 2.0$

File 1 is *bigger*, but file 2 is asked for far more often.

The Greedy Rule

Claim

Sorting by non-decreasing s_i/p_i is optimal.



Sorting by size alone, or by probability alone, is not enough. The **ratio** is what matters.

Exchange Argument

Suppose adjacent files a then b have $\frac{s_a}{p_a} > \frac{s_b}{p_b}$. First rewrite that:

$$\frac{s_a}{p_a} > \frac{s_b}{p_b} \implies p_b s_a > p_a s_b \implies p_a s_b - p_b s_a < 0.$$

Exchange Argument

Suppose adjacent files a then b have $\frac{s_a}{p_a} > \frac{s_b}{p_b}$. First rewrite that:

$$\frac{s_a}{p_a} > \frac{s_b}{p_b} \implies p_b s_a > p_a s_b \implies p_a s_b - p_b s_a < 0.$$

Now swap a and b :

- reading a costs s_b more, contributing $+p_a s_b$;
- reading b costs s_a less, contributing $-p_b s_a$;
- everything else is unaffected.

Exchange Argument

Suppose adjacent files a then b have $\frac{s_a}{p_a} > \frac{s_b}{p_b}$. First rewrite that:

$$\frac{s_a}{p_a} > \frac{s_b}{p_b} \implies p_b s_a > p_a s_b \implies p_a s_b - p_b s_a < 0.$$

Now swap a and b :

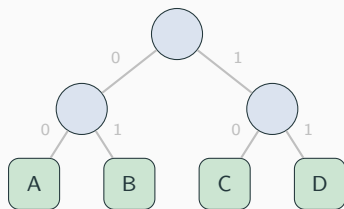
- reading a costs s_b more, contributing $+p_a s_b$;
- reading b costs s_a less, contributing $-p_b s_a$;
- everything else is unaffected.

The net change is $p_a s_b - p_b s_a < 0$, contradicting optimality. Again $O(n \log n)$.

Huffman Coding

Prefix Trees

A **prefix tree** is a full binary tree whose leaves are the symbols. Read the codeword off the root-to-leaf path: left is 0, right is 1.



$A \rightarrow 00$, $B \rightarrow 01$, $C \rightarrow 10$, $D \rightarrow 11$

This one is just fixed-length encoding, 2 bits per symbol.

Property

No codeword is a prefix of another.

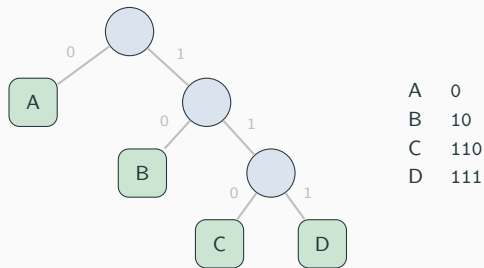
Why it holds automatically: if x 's codeword were a prefix of y 's, then x would lie on the path to y , making x an internal node.

But every symbol is a leaf. Contradiction.

This is exactly what makes decoding unambiguous.

Variable Length Can Do Better

If some symbols are much more frequent, give them shorter codewords.



AABABCD: fixed length needs $7 \times 2 = 14$ bits,
this tree needs $1+1+2+1+2+3+3 = 13$ bits.

Cost of a Prefix Tree

With symbols $1, \dots, n$ of frequencies $f_1, \dots, f_n$,

$$\text{cost}(T) = \sum_{i=1}^n f_i \cdot d_T(i)$$

where $d_T(i)$ is the depth of leaf i , that is the length of symbol i 's codeword.

Goal: find the tree T of minimum cost.

Huffman's Algorithm

The greedy idea: rare symbols get long codewords. Build the tree bottom up.

1. Make a leaf for each symbol i with key f_i ; put them all in a min-heap.
2. While the heap has more than one node:
 - extract the two smallest, u and v ;
 - create w with $f_w = f_u + f_v$ and children u, v ;
 - insert w back.
3. The last remaining node is the root.

$n - 1$ merges, each $O(\log n)$, so $O(n \log n)$.

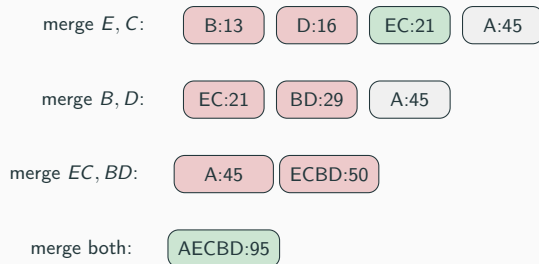
Worked Example

Symbol	A	B	C	D	E
Freq	45	13	12	16	9

heap:

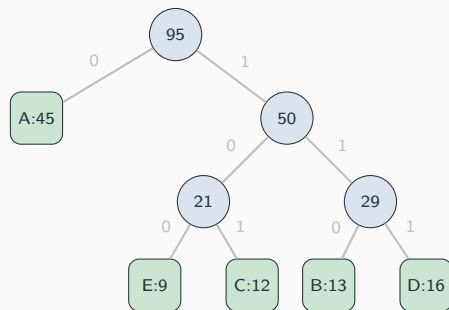


Merging



Red marks the two nodes pulled off the heap at each step.

The Resulting Tree



A	0
E	100
C	101
B	110
D	111

What It Saves

$$\text{cost}(T) = 45 \cdot 1 + 9 \cdot 3 + 12 \cdot 3 + 13 \cdot 3 + 16 \cdot 3 = 195 \text{ bits.}$$

A fixed-length code needs $\lceil \log_2 5 \rceil = 3$ bits per symbol:

$$3 \times 95 = 285 \text{ bits.}$$



The more skewed the frequencies, the bigger the win.

Encoding and Decoding

Encoding is a table lookup and concatenation:

$$A \ B \ C \ A \ D \ E \longrightarrow 0 \cdot 110 \cdot 101 \cdot 0 \cdot 111 \cdot 100 = 01101010111100$$

Encoding and Decoding

Encoding is a table lookup and concatenation:

$$A \ B \ C \ A \ D \ E \longrightarrow 0 \cdot 110 \cdot 101 \cdot 0 \cdot 111 \cdot 100 = 01101010111100$$

Decoding walks the tree. Start at the root, follow 0 left and 1 right; on reaching a leaf, emit the symbol and return to the root.



Prefix-freeness is what makes this walk unambiguous.

Exercise

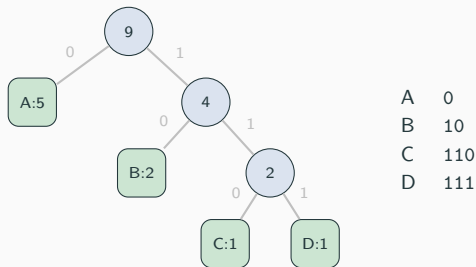
Build the Huffman tree for these frequencies and give the total cost.

Symbol	A	B	C	D
Freq	5	2	1	1

Compare it against a fixed-length code.

Solution

Merge $C:1$ and $D:1$ into $CD:2$, then $B:2$ with $CD:2$ into 4, then $A:5$ with that into 9.



$$\text{cost} = 5 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 + 1 \cdot 3 = 15 \quad \text{versus} \quad 2 \times 9 = 18 \text{ fixed length.}$$

Optimality: the Plan

Two steps.

1. A **structural lemma**: some optimal tree puts the two rarest symbols as siblings at maximum depth.
2. **Induction** on n : merging those two and recursing is exactly what Huffman does.

Sibling Lemma

Lemma

Let x, y be the two least frequent symbols. There is an optimal tree T^* in which x and y are siblings at maximum depth.

Sibling Lemma

Lemma

Let x, y be the two least frequent symbols. There is an optimal tree T^* in which x and y are **siblings at maximum depth**.

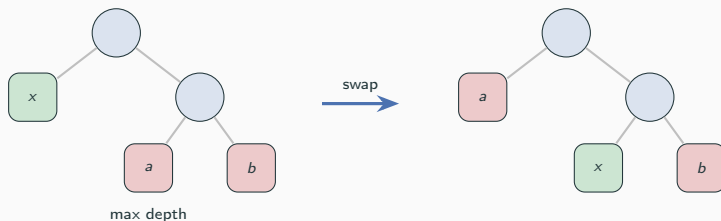
Proof. Take any optimal T^* and let a, b be siblings at maximum depth, with $f_a \leq f_b$. Since x, y are the two rarest, $f_x \leq f_a$ and $f_y \leq f_b$.

Swap x with a . The cost changes by

$$(f_x - f_a)(d_{T^*}(a) - d_{T^*}(x)) \leq 0,$$

since $f_x \leq f_a$ and a sits at maximum depth so $d_{T^*}(a) \geq d_{T^*}(x)$.

Sibling Lemma



So T' is no worse, hence also optimal. Swapping y with b the same way puts x and y together at maximum depth. $\square$

Theorem

Huffman's algorithm produces an optimal prefix tree.

Base case. $n = 2$: the only tree has both symbols at depth 1, and that is what Huffman builds.

Inductive step. Assume optimality for $n - 1$ symbols. Let x, y be the two rarest. By the lemma, some optimal T^* has them as siblings under a node w , so

$$d_{T^*}(x) = d_{T^*}(y) = d_{T^*}(w) + 1.$$

Theorem

Split the cost around w :

$$\begin{aligned}\text{cost}(T^*) &= \sum_{i \neq x, y} f_i d_{T^*}(i) + (f_x + f_y)(d_{T^*}(w) + 1) \\ &= \underbrace{\left[\sum_{i \neq x, y} f_i d_{T^*}(i) + (f_x + f_y) d_{T^*}(w) \right]}_{\text{cost}(T_w^*)} + (f_x + f_y).\end{aligned}$$

Here T_w^* is T^* with x, y removed and w made a leaf of frequency $f_x + f_y$: an instance with $n - 1$ symbols.

Theorem

Huffman does exactly this: merge x and y into w , recurse, then hang x and y back under w .

By the inductive hypothesis its tree T_w on the reduced instance satisfies $\text{cost}(T_w) \leq \text{cost}(T_w^*)$.
Therefore

$$\text{cost}(T_H) = \text{cost}(T_w) + (f_x + f_y) \leq \text{cost}(T_w^*) + (f_x + f_y) = \text{cost}(T^*).$$

Huffman's tree is at least as good as any optimal tree. ■